

Phase 2: Bubble Sort — Full Structured Beginner Note

1. Current Progress Before Phase 2

Before learning Bubble Sort, you already completed **Phase 1: Sorting Foundations and Evaluation Criteria**.

That means you should already know the basic words used to judge sorting algorithms:

Term	Simple meaning
Sorting	Arranging data in order
Ascending order	Smallest to largest
Descending order	Largest to smallest
Comparison	Checking two values to decide their order
Swap	Exchanging two values
Adaptiveness	Becoming faster when data is already sorted or nearly sorted
Extra memory	Extra space used by an algorithm
Stability	Keeping equal values in their original order
Big O	A way to describe how algorithm work grows
Pointer in C	A variable that stores an address

In Phase 1, you learned the vocabulary.

In Phase 2, you apply that vocabulary to your first real sorting algorithm: **Bubble Sort**.

2. Main Goal of Phase 2

The main goal of Phase 2 is to understand **Bubble Sort**.

Bubble Sort teaches these important ideas:

- Adjacent comparisons
- Passes
- Swaps
- Adaptive sorting using a flag
- Stability

- Best-case time
- Worst-case time
- Space complexity
- C implementation

Bubble Sort is not the fastest sorting algorithm, but it is very useful for beginners because it is easy to see how sorting happens step by step.

3. What Is Bubble Sort?

Bubble Sort is a sorting algorithm that compares neighboring elements and swaps them if they are in the wrong order.

Neighboring elements means elements that are side by side.

Example:

8 5

Here, 8 and 5 are neighbors.

If we want ascending order, we check:

8 > 5

This is true, so we swap them.

After swapping:

5 8

Bubble Sort repeats this process many times until the whole array becomes sorted.

4. Main Idea of Bubble Sort

Bubble Sort works like this:

1. Compare two neighboring elements.
2. If the left element is greater than the right element, swap them.
3. Move to the next neighboring pair.

4. Continue until the end of the unsorted part of the array.
5. Repeat the process for several passes.

The key comparison is:

$A[j] > A[j + 1]$

This means:

Is the current element greater than the next element?

If yes, the two values are in the wrong order and must be swapped.

5. Simple Example

Suppose we have:

8 5

We want ascending order.

Compare:

$8 > 5$

This is true.

So we swap:

5 8

Now the values are in correct ascending order.

6. Beginner Analogy

Think of large numbers as heavy stones.

In Bubble Sort, large numbers slowly move toward the end of the array.

Example:

8 5 7 3 2

The number 8 is the largest value.

During the first pass, 8 keeps swapping with smaller values until it reaches the end:

5 7 3 2 8

Now 8 is in its final sorted position.

Important idea:

Every pass pushes one large value to the end.

7. What Is a Pass?

A **pass** means one full scan through the unsorted part of the array.

Example array:

8 5 7 3 2

During the first pass, Bubble Sort compares neighboring values:

8 and 5
8 and 7
8 and 3
8 and 2

After these comparisons and swaps, the largest value reaches the end:

5 7 3 2 8

So after **pass 1**, the largest element is fixed.

After **pass 2**, the second largest element is fixed.

After **pass 3**, the third largest element is fixed.

General rule:

After passes, the largest elements are in their final positions.

8. Why the Largest Element Reaches the End

Suppose the array is:

8 5 7 3 2

The largest element is .

Bubble Sort compares neighbors:

8 and 5

Since > , swap:

5 8 7 3 2

Now compare:

8 and 7

Since > , swap:

5 7 8 3 2

Now compare:

8 and 3

Swap:

5 7 3 8 2

Now compare:

8 and 2

Swap:

5 7 3 2 8

The largest value moved step by step to the end.

9. Full Dry Run of Bubble Sort

Let us sort this array in ascending order:

8 5 7 3 2

Final answer should be:

2 3 5 7 8

10. Pass 1

Start:

8 5 7 3 2

Compare 8 and 5.

8 > 5, so swap:

5 8 7 3 2

Compare 8 and 7.

$8 > 7$, so swap:

5 7 8 3 2

Compare 8 and 3.

$8 > 3$, so swap:

5 7 3 8 2

Compare 8 and 2.

$8 > 2$, so swap:

5 7 3 2 8

After pass 1:

5 7 3 2 | 8

The 8 is fixed.

11. Pass 2

Now we ignore 8, because it is already in its correct final position.

Start:

5 7 3 2 | 8

Compare 5 and 7.

$5 > 7$ is false, so do not swap:

5 7 3 2 | 8

Compare 7 and 3.

7 > 3, so swap:

5 3 7 2 | 8

Compare 7 and 2.

7 > 2, so swap:

5 3 2 7 | 8

After pass 2:

5 3 2 | 7 8

The 7 and 8 are fixed.

12. Pass 3

Start:

5 3 2 | 7 8

Compare 5 and 3.

5 > 3, so swap:

3 5 2 | 7 8

Compare 5 and 2.

5 > 2, so swap:

3 2 5 | 7 8

After pass 3:

3 2 | 5 7 8

The 5, 7, and 8 are fixed.

13. Pass 4

Start:

3 2 | 5 7 8

Compare 3 and 2.

3 > 2, so swap:

2 3 | 5 7 8

Final sorted array:

2 3 5 7 8

14. Summary of the Dry Run

Original array:

8 5 7 3 2

After pass 1:

5 7 3 2 8

After pass 2:

5 3 2 7 8

After pass 3:

3 2 5 7 8

After pass 4:

2 3 5 7 8

Final sorted array:

2 3 5 7 8

15. Why Bubble Sort Needs $n - 1$ Passes

If an array has n elements, Bubble Sort needs at most $n - 1$ passes.

Example:

8 5 7 3 2

There are 5 elements.

So:

$n = 5$
 $n - 1 = 4$

Bubble Sort needs at most 4 passes.

Why not 5 passes?

Because after 4 elements are placed correctly, the remaining one element must already be in the correct position.

16. Why We Use $n - 1 - i$

In Bubble Sort, the inner loop is often written like this:

```
for (j = 0; j < n - 1 - i; j++)
```

The important part is:

```
n - 1 - i
```

This prevents unnecessary comparisons.

After every pass, one more element at the end is already sorted.

Example after pass 1:

```
5 7 3 2 8
```

The **8** is fixed.

So we do not need to compare with **8** again.

Example after pass 2:

```
5 3 2 7 8
```

Now **7** and **8** are fixed.

So we do not compare them again.

Meaning of variables:

Variable	Meaning
n	Number of elements
i	Current pass number
j	Current comparison position
n - 1 - i	Stops before the already sorted part

Important idea:

n - 1 - i avoids checking elements that are already in their final position.

17. Bubble Sort Pseudocode

Simple version:

```
repeat for each pass:
    compare neighboring elements
    if the left element is greater than the right element:
        swap them
```

Detailed version:

```
for i from 0 to n - 2:
    for j from 0 to n - 2 - i:
        if A[j] > A[j + 1]:
            swap A[j] and A[j + 1]
```

Explanation:

- `i` counts passes.
- `j` moves through neighboring elements.
- `A[j]` is the current element.
- `A[j + 1]` is the next element.
- If `A[j] > A[j + 1]`, they are swapped.

18. Basic Bubble Sort in C

This is the basic Bubble Sort idea without the adaptive flag:

```
#include <stdio.h>

void swap(int *x, int *y) {
    int temp = *x;
    *x = *y;
    *y = temp;
}

void BubbleSort(int A[], int n) {
    int i, j;

    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - 1 - i; j++) {
            if (A[j] > A[j + 1]) {
```

```

        swap(&A[j], &A[j + 1]);
    }
}
}

int main() {
    int A[] = {11, 13, 7, 12, 16, 9, 24, 5, 10, 3};
    int n = sizeof(A) / sizeof(A[0]);

    BubbleSort(A, n);

    for (int i = 0; i < n; i++) {
        printf("%d ", A[i]);
    }

    printf("\n");
    return 0;
}

```

Expected output:

```
3 5 7 9 10 11 12 13 16 24
```

19. Problem With Basic Bubble Sort

Basic Bubble Sort keeps running even if the array is already sorted.

Example:

```
3 7 9 10
```

This array is already sorted.

But basic Bubble Sort still performs passes and comparisons.

That wastes time.

To fix this, we use an adaptive version of Bubble Sort.

20. Adaptive Bubble Sort

An algorithm is **adaptive** if it becomes faster when the data is already sorted or nearly sorted.

Bubble Sort can become adaptive by using a variable called `flag`.

The flag tells us whether any swap happened during a pass.

If no swap happens during a full pass, the array is already sorted.

So the algorithm can stop early.

21. What Does `flag` Mean?

The variable `flag` is used to detect whether a swap happened.

At the start of each pass:

```
flag = 0;
```

This means:

No swap has happened yet in this pass.

If a swap happens:

```
flag = 1;
```

This means:

The array was not fully sorted during this pass.

At the end of the pass:

```
if (flag == 0) {  
    break;  
}
```

This means:

If no swap happened, the array is sorted, so stop the algorithm.

22. Adaptive Bubble Sort Dry Run

Example:

3 7 9 10

This array is already sorted.

Start pass 1:

flag = 0;

Compare 3 and 7.

No swap.

Compare 7 and 9.

No swap.

Compare 9 and 10.

No swap.

The pass ends and flag is still 0.

So Bubble Sort stops early.

This gives best-case time:

$O(n)$

23. Adaptive Bubble Sort C Program

```
#include <stdio.h>

void swap(int *x, int *y) {
    int temp = *x;
    *x = *y;
    *y = temp;
}
```

```

        *y = temp;
    }

    void BubbleSort(int A[], int n) {
        int i, j, flag;

        for (i = 0; i < n - 1; i++) {
            flag = 0;

            for (j = 0; j < n - 1 - i; j++) {
                if (A[j] > A[j + 1]) {
                    swap(&A[j], &A[j + 1]);
                    flag = 1;
                }
            }

            if (flag == 0) {
                break;
            }
        }
    }

    int main() {
        int A[] = {11, 13, 7, 12, 16, 9, 24, 5, 10, 3};
        int n = sizeof(A) / sizeof(A[0]);

        BubbleSort(A, n);

        for (int i = 0; i < n; i++) {
            printf("%d ", A[i]);
        }

        printf("\n");
        return 0;
    }

```

Expected output:

```
3 5 7 9 10 11 12 13 16 24
```

24. Explanation of the C Program

The program has three main parts:

1. `swap()` function
2. `BubbleSort()` function
3. `main()` function

25. The `swap()` Function

```
void swap(int *x, int *y) {  
    int temp = *x;  
    *x = *y;  
    *y = temp;  
}
```

This function swaps two integer values.

It uses pointers because it must change the real values inside the array.

Meaning:

Code	Meaning
<code>int *x</code>	<code>x</code> is a pointer to an integer
<code>int *y</code>	<code>y</code> is a pointer to an integer
<code>*x</code>	Value stored at the address inside <code>x</code>
<code>*y</code>	Value stored at the address inside <code>y</code>
<code>temp</code>	Temporary variable used to protect one value

When Bubble Sort calls `swap`, it calls it like this:

```
swap(&A[j], &A[j + 1]);
```

The `&` means address of.

So this gives the addresses of the two array elements.

26. The BubbleSort() Function

```
void BubbleSort(int A[], int n)
```

This function receives:

Parameter	Meaning
A[]	The array to sort
n	Number of elements in the array

27. Important Variables in Bubble Sort

Variable	Meaning
i	Counts passes
j	Compares neighboring elements
flag	Checks whether a swap happened
n	Number of elements
A[j]	Current element
A[j + 1]	Next element

28. Outer Loop Explanation

```
for (i = 0; i < n - 1; i++)
```

The outer loop controls the number of passes.

If there are `n` elements, Bubble Sort needs at most `n - 1` passes.

Example:

If there are 5 elements:

```
n = 5  
n - 1 = 4
```

So at most 4 passes are needed.

29. Inner Loop Explanation

```
for (j = 0; j < n - 1 - i; j++)
```

The inner loop compares neighboring values.

It checks:

```
A[j] > A[j + 1]
```

Examples of comparisons:

```
A[0] with A[1]  
A[1] with A[2]  
A[2] with A[3]  
A[3] with A[4]
```

The inner loop becomes shorter after each pass because the end part is already sorted.

30. Swap Condition Explanation

```
if (A[j] > A[j + 1])
```

This means:

If the left value is greater than the right value, they are in the wrong order.

So we swap them:

```
swap(&A[j], &A[j + 1]);
```

Important:

We use:

```
>
```

not:

```
>=
```

Because swapping equal values can break stability.

31. Flag Explanation in Code

At the start of each pass:

```
flag = 0;
```

This means no swap has happened yet.

If a swap happens:

```
flag = 1;
```

At the end of the pass:

```
if (flag == 0) {  
    break;  
}
```

If `flag` is still `0`, no swap happened, so the array is already sorted.

The `break` stops the loop early.

32. The `main()` Function

```
int main() {  
    int A[] = {11, 13, 7, 12, 16, 9, 24, 5, 10, 3};  
    int n = sizeof(A) / sizeof(A[0]);
```

```

BubbleSort(A, n);

for (int i = 0; i < n; i++) {
    printf("%d ", A[i]);
}

printf("\n");
return 0;
}

```

This function:

1. Creates an array.
2. Finds the number of elements.
3. Calls `BubbleSort()`.
4. Prints the sorted array.

33. Meaning of `sizeof(A) / sizeof(A[0])`

```
int n = sizeof(A) / sizeof(A[0]);
```

This finds the number of elements in the array.

Example:

```
int A[] = {11, 13, 7, 12, 16, 9, 24, 5, 10, 3};
```

There are 10 elements.

So:

```
n = 10
```

34. Bubble Sort Properties

Property	Bubble Sort result
Sorting type	Comparison-based sorting

Property	Bubble Sort result
Movement type	Swapping
Best-case time	$O(n)$ with adaptive flag
Worst-case time	$O(n^2)$
Space complexity	$O(1)$
Stable?	Yes
Adaptive?	Yes, if flag is used
In-place?	Yes
Good for huge data?	No
Good for learning?	Yes

35. Best-Case Time Complexity

The best case happens when the array is already sorted.

Example:

2 3 5 7 8

With the adaptive flag, Bubble Sort makes one pass.

No swaps happen.

The flag remains 0.

The algorithm stops early.

So best-case time is:

$O(n)$

This means the work grows directly with the number of elements.

36. Worst-Case Time Complexity

The worst case happens when the array is reverse sorted.

Example:

8 7 5 3 2

In this case, many swaps are needed.

Bubble Sort must perform many passes and many comparisons.

So worst-case time is:

$O(n^2)$

This means the work grows very quickly when the array becomes large.

37. Space Complexity

Bubble Sort sorts inside the original array.

It does not need another large array.

It only uses a few extra variables:

- `i`
- `j`
- `flag`
- `temp` inside `swap()`

So space complexity is:

$O(1)$

This means constant extra space.

38. Is Bubble Sort In-Place?

Yes.

Bubble Sort is an **in-place** sorting algorithm.

That means it sorts mainly inside the original array.

It does not require a second array to store the sorted result.

39. Is Bubble Sort Stable?

Yes.

Bubble Sort is stable if it swaps only when:

$$A[j] > A[j + 1]$$

It does not swap when the two values are equal.

40. Stability Example

Suppose we have:

5A 3 5B

5A and 5B have the same value.

But 5A came before 5B originally.

Bubble Sort compares equal values like this:

$$5 > 5$$

This is false.

So it does not swap them.

Final sorted result:

3 5A 5B

5A still comes before 5B.

So Bubble Sort is stable.

41. Why Using `>=` Breaks Stability

Correct condition:

```
if (A[j] > A[j + 1])
```

Wrong condition:

```
if (A[j] >= A[j + 1])
```

If we use `>=`, equal values may be swapped.

Example:

```
5A 5B
```

Using `>=`:

```
5 >= 5
```

This is true.

So the algorithm may swap them:

```
5B 5A
```

Now the original order is broken.

So the sort is no longer stable.

42. K-Pass Usefulness

Bubble Sort has a useful property:

After `k` passes, the `k` largest values are fixed at the end.

Example:

Original:

8 5 7 3 2

After 1 pass:

5 7 3 2 8

Largest value fixed: 8

After 2 passes:

5 3 2 7 8

Two largest values fixed: 7, 8

This means Bubble Sort can be useful if you only need the largest few elements.

Example:

If you need the top 3 largest values, you can run only 3 passes.

43. Why Bubble Sort Can Be Slow

Bubble Sort uses nested loops.

Nested loops often lead to $O(n^2)$ time.

Example with 5 elements:

8 5 7 3 2

It may do many comparisons and swaps.

For small arrays, this is acceptable.

For very large arrays, this becomes slow.

Example:

Sorting 5 numbers is easy.

Sorting 5,000,000 numbers with Bubble Sort would be very inefficient.

44. When Bubble Sort Is Useful

Bubble Sort is useful:

- For learning sorting
- For understanding comparisons
- For understanding swaps
- For understanding passes
- For small arrays
- For nearly sorted arrays when adaptive flag is used
- When you need only a few largest elements using a limited number of passes

Bubble Sort is not usually used for large real-world data because better algorithms exist.

45. Common Beginner Mistakes

Mistake 1: Forgetting to Reset `flag`

Wrong idea:

```
flag = 0; // only once before all passes
```

Correct idea:

```
for (i = 0; i < n - 1; i++) {  
    flag = 0;  
    ...  
}
```

The flag must be reset at the start of every pass.

If you do not reset it, early stopping may not work correctly.

Mistake 2: Using `>=` Instead of `>`

Wrong:

```
if (A[j] >= A[j + 1])
```

Correct:

```
if (A[j] > A[j + 1])
```

Using `>=` can swap equal values and break stability.

Mistake 3: Forgetting `n - 1 - i`

Less efficient:

```
for (j = 0; j < n - 1; j++)
```

Better:

```
for (j = 0; j < n - 1 - i; j++)
```

The first version may still sort correctly, but it wastes comparisons.

The second version avoids checking the already sorted end part.

Mistake 4: Thinking Bubble Sort Is Good for Huge Data

Bubble Sort is simple, but it is slow for large arrays.

Worst-case time:

```
O(n2)
```

So it is not a good choice for huge datasets.

Mistake 5: Confusing Passes With Comparisons

A pass is a full scan through the unsorted part.

A comparison is one check between two neighboring values.

One pass contains many comparisons.

Mistake 6: Forgetting to Use Addresses in `swap()`

Wrong:

```
swap(A[j], A[j + 1]);
```

Correct:

```
swap(&A[j], &A[j + 1]);
```

The swap function needs addresses so it can change the real array elements.

46. Key Vocabulary for Phase 2

Term	Simple meaning
Bubble Sort	Sorting by repeatedly comparing neighbors and swapping if needed
Adjacent elements	Elements that are side by side
Pass	One full scan through the unsorted part
Swap	Exchange two values
Adaptive Bubble Sort	Bubble Sort that stops early if no swap happens
Flag	A variable used to check whether a swap happened
Stable sort	Equal values keep their original order
In-place sort	Sorting happens inside the original array
Best case	The easiest input situation
Worst case	The hardest input situation
<code>O(n)</code>	Linear time

Term	Simple meaning
$O(n^2)$	Quadratic time
$O(1)$ space	Constant extra memory

47. Bubble Sort Evaluation Checklist

When you study Bubble Sort, ask these questions:

Comparisons

Does it compare values?

Yes.

It compares neighboring values:

```
A[j] > A[j + 1]
```

Movement

Does it move values?

Yes.

It moves values using swaps.

Adaptiveness

Can it stop early?

Yes, if we use the `flag` variable.

Memory

Does it need a large extra array?

No.

It uses $O(1)$ extra space.

Stability

Does it keep equal values in original order?

Yes, if we use `>` instead of `>=`.

K-pass result

What happens after `k` passes?

The `k` largest values are fixed at the end.

48. Complete Phase 2 Program With Comments

```
#include <stdio.h>

// Function to swap two integer values using pointers
void swap(int *x, int *y) {
    int temp = *x;
    *x = *y;
    *y = temp;
}

// Adaptive Bubble Sort function
void BubbleSort(int A[], int n) {
    int i, j, flag;

    // Outer loop controls passes
    for (i = 0; i < n - 1; i++) {
        flag = 0; // no swap has happened at the start of this pass

        // Inner loop compares neighboring elements
        for (j = 0; j < n - 1 - i; j++) {
            if (A[j] > A[j + 1]) {
                swap(&A[j], &A[j + 1]);
                flag = 1; // swap happened
            }
        }

        // If no swap happened, the array is already sorted
        if (flag == 0) {
            break;
        }
    }
}
```

```

int main() {
    int A[] = {11, 13, 7, 12, 16, 9, 24, 5, 10, 3};
    int n = sizeof(A) / sizeof(A[0]);

    BubbleSort(A, n);

    printf("Sorted array: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", A[i]);
    }

    printf("\n");
    return 0;
}

```

Expected output:

```
Sorted array: 3 5 7 9 10 11 12 13 16 24
```

49. Phase 2 Summary

Bubble Sort is the first real sorting algorithm in this curriculum.

The main idea is:

Compare neighboring elements. If they are in the wrong order, swap them.

The most important points are:

- Bubble Sort compares adjacent elements.
- It swaps when the left value is greater than the right value.
- One full scan is called a pass.
- After each pass, one large element is fixed at the end.
- After `k` passes, the `k` largest elements are fixed.
- `n - 1 - i` avoids comparing already sorted elements.
- The adaptive version uses a `flag`.
- If no swap happens in a pass, the array is already sorted.
- Best-case time is `O(n)` when the adaptive flag is used.
- Worst-case time is `O(n2)`.
- Space complexity is `O(1)`.
- Bubble Sort is stable if it uses `>` and not `>=`.
- Bubble Sort is good for learning, small arrays, and nearly sorted arrays.

- Bubble Sort is not good for huge datasets.
-

50. One-Sentence Memory Trick

Remember Bubble Sort like this:

Big values bubble toward the end, one pass at a time.

51. What Comes Next

After Bubble Sort, the next phase is **Phase 3: Insertion Sort**.

Bubble Sort moves values mainly by repeated neighboring swaps.

Insertion Sort uses a different idea:

Take one value and insert it into the correct place inside a sorted part.

So Phase 2 prepares you to compare Bubble Sort with Insertion Sort later.